

《Deep Reinforcement Learning from Human Preferences》中文逐段翻译

基于用户上传论文的中文整理版

原文题目: *Deep Reinforcement Learning from Human Preferences*

作者: Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, Dario Amodei

题目

基于人类偏好的深度强化学习

作者与单位

Paul F. Christiano (OpenAI), Jan Leike (DeepMind), Tom B. Brown, Miljan Martic (DeepMind), Shane Legg (DeepMind), Dario Amodei (OpenAI)

摘要

为了让复杂的强化学习系统能够在现实世界环境中有效地交互,我们需要向这些系统传达复杂目标。本文探索了一种目标定义方式:由人类(且不要求是专家)对两段轨迹片段进行偏好比较。我们表明,即使无法访问真实奖励函数,这种方法仍然能够有效求解复杂强化学习任务,包括 Atari 游戏和模拟机器人运动控制;与此同时,人类只需对智能体与环境交互过程中的不到 1% 提供反馈。这使得人类监督的成本显著降低,低到足以在当前最先进的强化学习系统中实际应用。为了展示该方法的灵活性,我们进一步说明:只需大约一小时的人类反馈,就可以成功训练出复杂的新颖行为。这些行为及其环境都明显比此前任何基于人类反馈学习到的任务更复杂。

1 引言

近些年,强化学习在大规模问题上的成功,主要出现在奖励函数明确定义的领域中,例如 Mnih 等 (2015, 2016) 以及 Silver 等 (2016) 的工作。不幸的是,许多任务的目标都十分复杂、定义模糊,或者难以明确写成奖励函数。若能突破这一限制,将会极大拓展深度强化学习的应用范围,也可能整体性地扩大机器学习的影响力。

例如,假设我们想用强化学习训练一个机器人去清理桌面,或者去炒鸡蛋。问题在于,如何构造一个合适的奖励函数并不清楚,因为这个奖励函数必须建立在机器人的传感器观测之上。我们当然可以尝试手工设计一个近似描述目标行为的简单奖励函数,但这往往会导致这样一种现象:系统优化了我们写下来的奖励函数,却并没有真正满足我们的偏好。近

期关于强化学习系统目标与人类价值错配的担忧，本质上就源于这一困难。如果我们能够成功地把真实目标传达给智能体，那将是缓解这些问题的重要一步。

如果我们掌握了目标任务的示范数据，那么可以通过逆强化学习提取奖励函数，再利用该奖励函数训练强化学习智能体。更直接地，也可以通过模仿学习去克隆示范行为。然而，对于那些人类本身就难以示范的行为，这些方法并不直接适用，例如控制一个具有大量自由度且形态与人类完全不同的机器人。

另一种思路，是允许人类对系统当前的行为提供反馈，再利用这些反馈来定义任务。从原理上看，这仍然属于强化学习范式；但如果直接把人类反馈当作奖励函数，那么对于需要数百乃至数千小时经验的强化学习系统来说，代价会高得难以承受。因此，若想真正用人类反馈去训练深度强化学习系统，就必须把所需反馈量降低几个数量级。

本文的方法是：先从人类反馈中学习一个奖励函数，再去优化这个奖励函数。这个总体思想此前曾被考虑过，但本文真正面对了将其扩展到现代深度强化学习时遇到的规模化挑战，并展示了迄今为止最复杂的一批“由人类反馈学得”的行为。

概括来说，我们希望为“没有明确奖励函数的序列决策问题”找到一种解决方案，使其满足以下四点：第一，我们只需能识别出理想行为，而不一定必须亲自示范它；第二，非专家用户也能够教会智能体完成任务；第三，这种方法可以扩展到大规模问题；第四，它对用户反馈的消耗足够节省。

本文算法的核心是：一边让智能体学习符合人类偏好的奖励函数，一边同步训练策略去优化当前预测得到的奖励函数。我们要求人类比较的是“短视频片段”而不是给出绝对数值评分。我们发现，在若干任务领域中，比较两个片段对人来说更容易，而且对学习人类偏好同样有效。

比较短视频片段的速度，几乎与比较单个状态一样快；但本文表明，前者能提供明显更有价值的信息。此外，我们还发现，在线收集反馈不仅能提升系统性能，也能减少智能体利用学习到的奖励函数漏洞进行“投机取巧”的风险。

本文实验覆盖两个领域：一是 Arcade Learning Environment 中的 Atari 游戏，二是 MuJoCo 物理模拟器中的机器人控制任务。实验表明，即使真实奖励函数对算法不可见，只要从非专家人类那里收集少量反馈——少则十五分钟，多则五小时——就足以学会大多数原始强化学习任务。随后，作者又研究了每个领域中的新行为，例如后空翻或按车流方向驾驶。结果显示，即使这些行为很难通过手工方式设计奖励函数加以鼓励，算法依然能在约一小时的人类反馈下将其学出来。

1.1 相关工作

强化学习从人类评分或排序中学习，已经有一条较长的研究脉络，包括 Akrouer 等 (2011, 2012)、Pilarski 等 (2011)、Wilson 等 (2012)、Sugiyama 等 (2012)、Wirth 和 Fürnkranz (2013)、Daniel 等 (2015)、El Asri 等 (2016)、Wang 等 (2016) 以及 Wirth 等 (2016) 的工作。还有一些研究更一般地讨论“基于偏好而非绝对奖励值的强化学习”，以及在强化学习以外的场景中利用人类偏好进行优化。

本文算法在基本思想上与 Akrouer 等 (2012, 2014) 最为接近。他们研究的是四自由度连续控制任务和较小的离散任务，并且可以假设奖励是手工特征期望的线性函数。相比之

下，本文处理的是拥有数十个自由度的物理控制任务，以及完全没有手工特征的 Atari 任务。环境复杂度的差异迫使作者采用不同的强化学习算法与奖励模型，同时还要应对不同的算法权衡。一个显著区别是：Akroun 等工作让人类对“整条轨迹”做偏好判断，而本文让人类比较“短片段”。因此，虽然本文收集的比较数量多了大约两个数量级，但所需人类时间却只增加了不到一个数量级。其他差异则集中在训练流程的改造上，例如异步训练与模型集成，以便适配非线性奖励模型和现代深度强化学习。

本文关于反馈收集的方式与 Wilson 等 (2012) 比较接近。不过，Wilson 等假设奖励函数等于到某个未知“目标策略”的距离，而该目标策略本身又是手工特征的线性函数。他们利用贝叶斯推断拟合奖励函数，并不是通过强化学习优化策略，而是直接使用目标策略的最大后验估计来生成轨迹。其实验中的“人类反馈”也是从贝叶斯模型中合成出来的，而本文则真正使用非专家用户提供的真实反馈。Wilson 等方法能否扩展到复杂任务、能否适用于真实人类反馈，并不明确。

MacGlashan 等 (2017)、Pilarski 等 (2011)、Knox 和 Stone (2009) 以及 Knox (2012) 都研究了“从真实人类反馈进行强化学习”，尽管它们在算法层面与本文的相似度较低。在 MacGlashan 等和 Pilarski 等的设定里，只有当人类训练者正在提供反馈时，学习才会发生。这对于像 Atari 这样需要数千小时经验才能学出高质量策略的领域来说，显然不可行；即便对本文讨论的最简单任务，其成本也会过高。TAMER 框架同样学习奖励函数，但其应用情形要简单得多，因为目标策略可以在相对很短时间内学得。

本文也可以看作 cooperative inverse reinforcement learning 框架的一个特例。该框架把问题视为一个人与机器人共同参与的二人博弈，双方通过与环境交互来最大化人的奖励函数。而在本文场景中，人类参与这场博弈的唯一方式，就是表达自己对不同轨迹片段的偏好。

与此前所有工作相比，本文最关键的贡献在于：首次将人类反馈有效扩展到深度强化学习的尺度，并成功学习到了显著更复杂的行为。这也与近年来“把奖励学习方法扩展到大型深度学习系统”的趋势相一致，例如逆强化学习、模仿学习、半监督技能泛化以及利用示范引导强化学习等方向。

2 预备知识与方法

2.1 问题设定与目标

本文考虑一个智能体与环境按时间步交互的过程：在每个时刻 t ，环境向智能体提供一个观测 $o_t \in O$ ，随后智能体向环境发送动作 $a_t \in A$ 。

在传统强化学习中，环境还会提供奖励 $r_t \in R$ ，而智能体的目标是最大化折扣累计回报。本文不再假设环境能给出奖励信号，而是假设存在一个人类监督者，他可以在两段轨迹片段之间表达偏好。所谓轨迹片段，是由一段观测与动作组成的序列：

$$\sigma = ((o_0, a_0), (o_1, a_1), \dots, (o_{k-1}, a_{k-1})) \in (O \times A)^k.$$

记 $\sigma^1 \succ \sigma^2$ 表示人类更喜欢轨迹片段 σ^1 而不是 σ^2 。直观来说，智能体的目标是生成尽可

能被人类偏好的轨迹，同时尽量减少向人类发起查询的次数。

更精确地，本文从两个角度评估算法行为。

定量评估：若人类偏好 $\succ$ 可以由某个奖励函数 $r: O \times A \rightarrow \mathbb{R}$ 生成，也就是说，当且仅当第一段片段上逐步奖励之和大于第二段片段上逐步奖励之和时，人类更偏好前者，那么一个好的智能体就应当在该奖励函数下取得较高总回报。因此，如果真实奖励函数已知，就可以进行定量评估。理想情况下，本文方法的表现应当接近直接用强化学习去优化真实奖励函数时的水平。

定性评估：有时我们根本没有一个可供定量评估的奖励函数——这正是本文方法真正有实际价值的情形。在这类情况下，我们只能从定性角度去判断：智能体是否较好地满足了人类偏好。本文在这类实验中，会先用自然语言描述任务目标，让人类根据该目标来评价智能体行为，最后展示智能体尝试完成该目标的视频。

本文基于轨迹片段比较的模型，与 Wilson 等 (2012) 中的轨迹偏好查询非常相似，但有一个重要区别：本文不假设能够把系统重置到任意状态，因此被比较的两个片段通常来自不同的起始状态。这会让人类比较的解释变得更复杂，但实验表明，即便评分者并不了解算法原理，本文方法仍然能够克服这一困难。

2.2 本文方法

在任意时刻，本文方法维护两个由深度神经网络参数化的对象：策略 $\pi: O \rightarrow A$ 与奖励函数估计 $\hat{r}: O \times A \rightarrow \mathbb{R}$ 。

这两个网络通过三个并行过程持续更新。

第一，策略 π 与环境交互，产生一组轨迹 $\{\tau^1, \dots, \tau^i\}$ 。随后使用传统强化学习算法更新 π 的参数，使其最大化预测奖励 $r_t = \hat{r}(o_t, a_t)$ 的累积和。

第二，从这些轨迹中选出若干片段对 (σ^1, σ^2) ，发送给人类进行比较。

第三，利用监督学习，根据迄今为止收集到的人类比较数据来优化 $\hat{r}$ 的参数。

这三个过程是异步运行的：轨迹从过程一流向过程二，人类比较结果从过程二流向过程三，而更新后的奖励模型参数又从过程三反馈给过程一。后续小节将分别介绍这三个过程。

2.2.1 策略优化

一旦用 $\hat{r}$ 计算出了奖励，剩下的就是一个标准强化学习问题。理论上，只要适合当前领域，任何强化学习算法都可以使用。这里有一个微妙之处：奖励函数 $\hat{r}$ 可能是非平稳的，因此更适合选择那些对奖励变化较为鲁棒的方法。出于这一考虑，作者重点采用了策略梯度类算法，因为这类方法在相似问题上已经取得成功。

在本文中，作者使用 advantage actor-critic (A2C) 来训练 Atari 游戏智能体，使用 trust region policy optimization (TRPO) 来训练模拟机器人控制任务。各自的参数设置基本沿用传统强化学习任务中已经证明有效的默认配置。唯一被特别调整的超参数，是 TRPO 的熵奖励项。这是因为 TRPO 依赖 trust region 保证足够探索，而在奖励函数不断变化时，默认探索强度可能不够。

作者还对 $\hat{r}$ 产生的奖励进行了归一化处理, 使其具有零均值和恒定标准差。这是很常见的预处理方式, 在本文中特别合理, 因为仅凭偏好学习问题本身, 并不能唯一确定奖励函数的绝对平移位置。

2.2.2 偏好收集

人类监督者会看到两个轨迹片段的可视化结果, 形式为两个简短视频。在本文所有实验中, 这些视频片段的时长都在 1 到 2 秒之间。

随后, 人类需要指出: 自己更喜欢哪一个片段; 或者认为两者一样好; 或者表示自己无法比较。

所有人类判断被记录在数据库 D 中, 其元素为三元组 $(\sigma^1, \sigma^2, \mu)$, 其中 σ^1 和 σ^2 是两个片段, μ 是定义在 $\{1, 2\}$ 上的分布, 用以表示用户偏好哪个片段。如果人类明确选择一个片段更好, 那么 μ 的全部概率质量就放在对应选项上; 如果人类认为两者一样好, 则 μ 为均匀分布; 如果人类认为无法比较, 那么该样本不会被纳入数据库。

2.2.3 奖励函数拟合

如果把奖励函数估计 $\hat{r}$ 看作“偏好预测器”, 那么可以把它理解成解释人类判断的潜在因子, 并假设: 人类偏好某个片段的概率, 与该片段整个时长上的潜在奖励和呈指数关系。于是,

$$\hat{P}[\sigma^1 \succ \sigma^2] = \frac{\exp \sum_t \hat{r}(o_t^1, a_t^1)}{\exp \sum_t \hat{r}(o_t^1, a_t^1) + \exp \sum_t \hat{r}(o_t^2, a_t^2)}.$$

随后, 作者最小化该预测分布与真实人类标签之间的交叉熵损失:

$$\text{loss}(\hat{r}) = - \sum_{(\sigma^1, \sigma^2, \mu) \in D} \mu(1) \log \hat{P}[\sigma^1 \succ \sigma^2] + \mu(2) \log \hat{P}[\sigma^2 \succ \sigma^1].$$

这种建模方式遵循 Bradley-Terry 成对比较模型, 本质上也可以理解为 Luce-Shephard 选择规则在轨迹片段偏好上的一个特例。它可以类比国际象棋中的 Elo 评分系统: 正如两位棋手 Elo 分差决定一方战胜另一方的概率, 两个轨迹片段之间的预测奖励差, 也决定了人类偏好前者而非后者的概率。

在这一基本框架之上, 作者还加入了若干在早期实验中证明有用的改动, 并在第 3.3 节中进行了分析。

其一, 作者训练的是一个预测器集成。每个预测器都从数据库 D 中有放回抽样得到 $|D|$ 个三元组进行训练。最终的 $\hat{r}$ 则通过对每个预测器先独立归一化、再求平均得到。

其二, 对每个预测器, 都拿出约 $1/e$ 的数据作为验证集。训练中使用 ℓ_2 正则, 并动态调节正则系数, 使验证损失保持在训练损失的 1.1 到 1.5 倍之间。在某些领域中, 还额外使用 dropout 进行正则化。

其三, 作者没有直接使用式中的 softmax, 而是假设人类有 10% 的概率会随机作答。直观上, 这是因为真实评分者始终存在一个不会随着奖励差异无限增大而降为零的出错概率。

2.2.4 查询选择

作者基于奖励函数估计器的不确定性近似，来决定下一步该收集哪些偏好。这一做法与 Daniel 等 (2014) 的思想相似：首先采样大量长度为 k 的轨迹片段对，再让集成中的每个奖励预测器分别判断哪一个片段更优，最后选取那些在集成成员之间预测方差最大的片段对，交给人类进行比较。

作者指出，这种做法仍然只是一个较粗糙的近似；第 3 节中的消融实验甚至表明，在某些任务中它还会损害性能。理想情况下，更合理的标准应当是每个查询的期望信息价值，但这部分内容被留作未来工作。

3 实验结果

作者使用 TensorFlow 实现了整套算法，并通过 OpenAI Gym 接入 MuJoCo 以及 Arcade Learning Environment。

3.1 不可观测奖励下的强化学习任务

在第一组实验中，作者尝试在看不到真实奖励的前提下，解决一系列深度强化学习基准任务。智能体对于任务目标的理解，完全来自“向人类询问两段轨迹谁更好”这一过程。实验目标是在合理时间内、使用尽可能少的查询次数完成任务。

在这些实验中，反馈由外部承包人员提供。实验前，他们会先收到每个任务 1 到 2 句话的简要说明，然后再去比较数百到数千对轨迹片段。每个片段时长都在 1 到 2 秒之间。承包人员平均每个查询需要 3 到 5 秒，因此整个真实人类反馈实验大约需要 30 分钟到 5 小时的人类时间。

为便于比较，作者还设计了一个“合成神谕”。该神谕对轨迹的偏好完全由底层任务的真实奖励决定：当智能体发起一次比较请求时，不再交给真人，而是立即根据哪一段片段实际累积奖励更高来返回偏好。作者还把“直接使用真实奖励进行强化学习”作为基线。这里的目标不是超越 RL，而是在看不到真实奖励、只依赖更稀缺反馈的情况下做到尽量接近 RL。不过需要注意，真实人类反馈在某些情况下确实有可能优于标准 RL，因为人类可能给出更具塑形作用的奖励信号。

文中具体实验细节，包括模型结构、环境修改方式以及用于优化策略的强化学习算法，都被放在附录 A 中说明。

3.1.1 模拟机器人任务

作者首先考虑了八个 MuJoCo 模拟机器人任务，这些任务来自 OpenAI Gym。为了避免环境本身把任务信息“偷偷编码”给算法，作者对这些任务做了少量修改，具体细节见附录 A。这些任务的奖励函数通常是距离、位置和速度等特征的线性组合，并进一步是这些特征的二次函数。作者还加入了一个简单的 cartpole 任务（文中记为 pendulum）作为对照，因为这类任务与早期相关工作中的复杂度更为接近。

实验结果表明，当仅使用 700 次真人标注时，本文方法在这些任务上几乎都能达到传统强化学习的水平。与真实奖励相比，使用学习得到的奖励函数训练，通常稳定性更差、方差更高，但平均性能大致相当。

更令人意外的是，当标注量增至 1400 条时，本文算法的表现甚至略好于直接使用真实奖励训练。这可能是由于学到的奖励函数具有更好的塑形效果——在高回报行为之前常见的那些行为，也会被赋予一定正奖励。

整体来看，真实人类反馈通常只比合成反馈略差一些；不同任务中，人类反馈的效率大致在“真实奖励神谕的一半”到“与其相当”之间波动。在 Ant 任务中，真实人类反馈显著优于合成反馈，原因似乎在于作者要求人类偏好“机器人保持直立”的轨迹，而这种偏好本身就构成了有益的奖励塑形。虽然 RL 原始奖励里也包含鼓励机器人保持直立的项，但简单手工设计的这一项并没有人类偏好那样有效。

3.1.2 Atari 任务

第二组实验是七个 Atari 游戏，这些游戏与 Mnih 等（2013）中使用的集合相同。

结果显示，在这些更具挑战性的环境中，本文方法要想全面匹配传统 RL 会更困难；不过，在大多数游戏中，算法仍然表现出显著学习能力，并且在部分任务上与 RL 持平甚至超过 RL。具体来说，在 BeamRider 和 Pong 上，即使只用 3300 条左右的合成标签，也已经能够达到或接近 RL 的表现；在 Seaquest 和 Qbert 上，合成反馈最终也能接近 RL，但学习速度更慢；在 SpaceInvaders 和 Breakout 上，合成反馈始终没能追平 RL，但智能体仍然有明显提升，例如在 SpaceInvaders 中常常能打过第一关，在 Breakout 中可达到 20 分左右，标签更多时甚至能到 50 分。

在大多数游戏上，真实人类反馈的表现与相同标注量下的合成反馈相近，或略差一些；很多时候，它大致相当于“少 40% 标签量”的合成反馈。这可能来自多个因素：人类标注本身会出错，不同承包者对同一轮实验的打标标准不完全一致，以及实际打标速度不均匀，导致标签过度集中在状态空间的某些狭窄区域。作者认为，这类问题未来可以通过改进众包打标流程来缓解。

在 Qbert 上，使用真实人类反馈时，本文方法甚至没能学会打过第一关；原因可能在于，Qbert 的短视频片段本身就比较难理解、也更难评价。Enduro 则体现了另一个特点：A3C 由于随机探索很难做到成功超车，因此该游戏本身就难学，相应地用合成标签也学得较差；而真人标注者往往会奖励任何“向超车前进”的迹象，这实际上相当于对奖励做了塑形，因此在该游戏中反而超过了 A3C，其结果与 DQN 的水平相近。

3.2 新颖行为

传统强化学习任务的实验有助于判断方法是否有效，但人机交互的终极目标，是去解决那些根本没有现成奖励函数的任务。在与前述实验相同的参数设置下，作者展示了算法可以学会若干复杂的新行为。

第一，让 Hopper 机器人连续做后空翻。该行为使用 900 次查询、不到一小时的人类反馈完成训练。最终智能体能够稳定地完成一个后空翻、落地保持直立，并继续重复这一动作。

第二，让 Half-Cheetah 机器人单腿支撑向前移动。这个行为使用 800 次查询、不到一小时完成训练。

第三，在 Enduro 中保持与其他车辆并排行驶。该行为大约使用 1300 次查询以及 400 万帧环境交互训练得到；最终智能体可以在一段相当长的时间里几乎与其他移动汽车严格保持齐平，尽管在背景变化时仍会出现混乱。

这些行为的视频都由作者给出的链接提供，且这些实验中的反馈均由论文作者本人完成。

3.3 消融实验

为了更好地理解算法各组成部分的作用，作者设计了一组变体进行比较。

第一，查询不再按分歧度优先，而是均匀随机抽取。

第二，不再使用预测器集成，而只训练单个奖励预测器。在这种设定下，由于无法再利用集成估计分歧，查询也改为随机选择。

第三，只在训练开始时收集查询，不再在整个训练过程中持续在线收集。

第四，去掉 ℓ_2 正则，只保留 dropout。

第五，仅在机器人控制任务中，把轨迹片段长度缩短为 1，也就是不再使用片段比较。

第六，不再通过比较来拟合 $\hat{r}$ ，而是假设有一个神谕能直接提供整段轨迹片段的真实总奖励，然后用均方误差去拟合这些总奖励。

MuJoCo 与 Atari 的消融结果表明，其中最值得关注的是“离线训练奖励预测器”表现很差。由于状态占据分布本身是非平稳的，预测器往往只能学到真实奖励的一部分，而一旦智能体去最大化这个不完整奖励，就容易出现从真实奖励角度看明显不理想、甚至怪异的行为。例如在 Pong 中，离线训练有时会让智能体只学会“避免丢分”，却不会主动“得分”；这样会导致双方长期来回击球，几乎无限重复相同事件序列。这说明，在一般情况下，人类反馈必须与强化学习过程交织进行，而不能静态、一次性地提前给定。

作者之所以更倾向于收集比较而不是绝对分数，主要原因是：在连续控制任务以及新颖行为任务中，让人类给出一致的比较判断，明显比给出一致的绝对评分容易得多。但即便如此，比较与打分对性能的影响仍值得分析。实验发现，在连续控制任务中，“预测比较结果”明显优于“预测分数”。一个可能原因是，奖励尺度在这些任务中变化很大，使回归问题变得困难，而只学习比较关系会显著平滑这一难度。在 Atari 中，作者对奖励做了截断，实际上主要预测其符号，从而避免了这类问题；不过这对连续控制任务并不适用，因为这些任务中相对奖励幅值本身就很重。在 Atari 中，比较与目标值监督的性能差异显著，但二者都没有始终一致地胜出。

作者还观察到：如果把视频片段换成单帧图像，性能会有很大下降。若想用单帧达到同样效果，必须收集显著更多的比较样本。整体上，更长的片段在“每个片段的信息量”上显著更有效，但在“每帧的信息量”上则略差。作者认为，对于很短的片段，人类要花一部分时间先理解场景；而片段更长时，评价时间大致与片段长度呈线性关系。于是，实验中选择的是“刚好进入线性区”的最短片段长度。在 Atari 中，较长片段也更容易比较，因为它们比单帧提供了更多上下文信息。

4 讨论与结论

在许多任务里，智能体与环境的交互成本远低于人与系统交互的成本。本文表明，通过单独训练一个奖励模型，并用监督学习方式学习该模型，可以把“人类交互复杂度”降低大约三个数量级。这不仅说明：我们确实可以依靠人类偏好来有效训练深度强化学习智能体；同时也说明，在样本复杂度继续下降方面，我们已经开始遇到收益递减，因为此时计算资源的成本已经与非专家人类反馈的成本处于同一量级。

尽管偏好收集与未知奖励下的强化学习已有大量研究，本文首次提供了证据，说明这些技术可以经济地扩展到最先进的深度强化学习系统上。这是将深度强化学习应用到复杂现实任务的重要一步。

未来工作有望进一步提高从人类偏好中学习的效率，并把该方法扩展到更广泛的任务范围。

从长远来看，理想目标是：让“从人类偏好中学习一个任务”不比“从程序化奖励信号中学习一个任务”更困难。只有这样，强大的强化学习系统才能真正服务于复杂的人类价值，而不是仅仅服务于低复杂度、容易形式化的目标。

附录 A：实验细节（翻译）

许多强化学习环境中，episode 的终止条件与智能体行为直接相关，例如智能体死亡或摔倒时就结束一局。作者发现，即使奖励函数不可见，这类终止条件本身也会向智能体泄露任务信息。为了避免这种微妙的监督来源干扰“仅从人类偏好学习”的结论，作者去除了所有可变长度 episode。

在 Gym 机器人任务中，原始环境会在某些参数超出预设范围时结束 episode，例如机器人摔倒。作者把这些终止条件改写成惩罚项，用来鼓励参数保持在合法范围内，而这一规律也需要智能体自己学会。

在 Atari 中，作者不再把失去生命或 episode 结束的信号传给智能体（虽然环境内部仍然会真正重置），等价于把环境视为一条连续不断的长 episode。在使用合成神谕反馈时，除 Pong 外，所有游戏中的 episode 结束也被替换为惩罚，智能体需要自己学会理解这一惩罚。

去除可变长度 episode 以后，智能体就只剩下环境本身所包含的信息，而关于“应该做什么”的唯一指引便来自人类反馈。

在训练开始阶段，作者会先从未经训练、随机初始化的策略 rollout 中抽取若干轨迹片段进行比较。在 Atari 领域中，作者还会在 RL 开始前，先用 200 个 epoch 预训练奖励预测器，以降低系统基于一个尚未训练好的预测器而不可逆地学出糟糕策略的风险。其后，标签输入速率会随着时间步数增加而按反比衰减，也就是所谓的 label annealing，用来平衡“早期需要一个足够好的预测器”和“随着 RL 学到新状态，预测器也必须持续适应”这两方面需求。真实人类反馈实验中，作者也尝试采用类似衰减策略，但由于承包者打标速度不均匀，实际只能近似实现。

除非特别说明，实验都使用由 3 个预测器组成的集成；在查询选择时，会先采样 10 倍于最终展示数量的片段对候选，再依据不同预测器之间的最大方差来选择真正展示给人类

的样本。

A.1 模拟机器人任务

OpenAI Gym 的连续控制任务会惩罚过大的力矩。但由于力矩并不是人类监督者能直接观察到的量，所以这种奖励并不能很好代表人类对于轨迹的偏好，因此作者将其去除。

在机器人任务中，作者使用 TRPO 优化策略，折扣因子设为 $\gamma = 0.995$ ，广义优势估计中的 $\lambda = 0.97$ 。奖励预测器是一个两层全连接网络，每层 64 个隐藏单元，激活函数采用 leaky ReLU，负斜率为 0.01。用于比较的轨迹片段时长为 1.5 秒，根据任务不同，对应 15 到 60 个时间步。

作者将奖励预测归一化到标准差为 1。在利用奖励预测器训练策略时，除 swimmer 外，所有任务都加入 0.01 的熵奖励；而 swimmer 使用 0.001。这是因为在奖励函数不断变化的情况下，需要更强探索来应对非平稳性。

训练开始时，有 25% 的比较样本来自随机初始化策略；之后在训练进行到 T 帧时，打标速率按照 $2 \times 10^6 / (T + 2 \times 10^6)$ 衰减。

A.2 Atari

Atari 实验沿用了 Mnih 等 (2015) 中的标准环境包装：episode 开始时随机执行 0 到 30 个 no-op；相邻帧做 max-pooling；堆叠 4 帧作为状态；frameskip 为 4；失去生命时结束 episode（但不真正重置环境）；奖励裁剪到 $[-1, 1]$ 。

Atari 游戏画面通常会显示分数，而这在理论上可能使系统直接从画面中推断奖励。为了把注意力放在“从复杂游戏动态中推断奖励”上，作者把七个游戏中的记分区域都统一替换成黑色背景；其中 BeamRider 额外遮住了敌机计数，Enduro 额外遮住了速度表。

Atari 策略采用同步形式的 A3C，也就是 A2C，网络结构与 Mnih 等 (2015) 一致。超参数基本采用标准设置：熵奖励系数 $\beta = 0.01$ ，学习率 0.0007，并在 8000 万时间步内线性衰减到 0；每次更新使用 $n = 5$ 步；并行 worker 数为 16；折扣因子 $\gamma = 0.99$ ；策略梯度优化器为 Adam，并采用文中给出的动量参数。

奖励预测器输入为 84×84 图像，并堆叠 4 帧形成 $84 \times 84 \times 4$ 的张量。该输入经过 4 层卷积，卷积核大小依次为 7×7 、 5×5 、 3×3 、 3×3 ，步幅依次为 3、2、1、1，每层 16 个卷积核，并使用 leaky ReLU。随后接一个 64 维全连接层，最后输出一个标量。所有卷积层都使用 batch normalization 和 dropout（系数 0.5）来防止过拟合；同时还采用了自适应 ℓ_2 正则。

由于奖励预测器最终只用于比较两个时间序列求和后的大小，因此其绝对尺度并不重要。作者将其归一化到标准差 0.05，这样就无需为了预测奖励而重新改动学习率和熵奖励等参数。

Atari 中的轨迹片段长度为 25 个时间步，大约相当于在 15 fps、带帧跳条件下的 1.7 秒视频。训练开始时，作者先从随机初始化策略中收集 500 个比较样本；在训练进行到 T 帧后，打标速率每经过 5×10^6 帧下调一次，总体上近似与 $5 \times 10^6 / (T + 5 \times 10^6)$ 成比例。

预测器与 RL 智能体异步训练；在作者的硬件环境中，预测器大约每处理 1 条标签，对

应 RL 前进 10 个时间步。作者只保留最近 3000 条标签构成缓冲区，并在其上持续循环训练，以保证当标签总数变大后，预测器依然能充分重视新标签，因为这些新标签往往反映了状态分布的转移。

附录 B：给标注人员的说明（摘译）

B.1 MuJoCo

标注人员需要先在表格中登记一个时间段，然后打开给定网址。系统会不断展示两个 AI 控制虚拟机器人的视频片段。标注者需要选择“哪一个片段中发生了更好的事情”，而且只根据自己在片段里实际看到的内容做判断。

不同任务对应不同的评价准则。例如 Hopper 与 Walker 主要看机器人中心是否向右移动，若接近则再比较其高度；Swimmer 与 Cheetah 则主要看向右移动速度；Ant 先看是否保持直立，其次看高度，再比较向右速度；Reacher 看绿色点是否尽可能接近红点；Pendulum 和 Double-pendulum 则主要看是否保持竖直，以及小车是否靠近轨道中心。

如果两个片段看起来差不多，可以选择 tie；如果看不懂或难以判断，则选择 can't tell。系统还支持键盘快捷键：左右箭头选择左右视频，上箭头表示平局，下箭头表示无法判断。

B.2 Atari

在 Atari 标注任务中，作者要求标注者先亲自玩该游戏 5 分钟，以便理解玩法。因为如果从未玩过，仅凭短片段往往很难判断游戏目标是什么。

之后，标注者同样需要在两个 AI 游戏视频片段之间做选择，挑出“发生了更好事情”的那一个。只根据片段中实际看到的行为判断，而不要去考虑智能体此前是怎么进入该状态的，例如它当前是否有更多生命、是否已经进入更高关卡等。

不同游戏的好坏标准也被明确写出，例如 BeamRider 和 SpaceInvaders 中，击中敌人是好事、自己被击中是坏事；Breakout 中要接住球并打掉砖块；Enduro 中要超车且避免被超；Pong 中要把球打过对手球拍；Qbert 中要尽可能改变方块颜色，同时避免掉下边缘或撞上敌人；SeaQuest 中要击中鱼和敌方潜艇、营救潜水员，同时避免缺氧或被击中；Enduro 的 even 模式则要求尽量与其他车保持齐平。

如果两个片段差不多，选择 tie；如果真的看不懂，再选择 can't tell。作者还特别提醒：尽量减少 can't tell 的使用，只有在确实困惑时才选择它。

参考文献说明

原论文后续参考文献条目较长，本文档未逐条翻译其文献表述，建议在阅读时与原文对应查看；正文内容与附录说明已完成中文整理。

说明：本中文稿依据用户上传论文内容整理而成，便于阅读与学习使用。